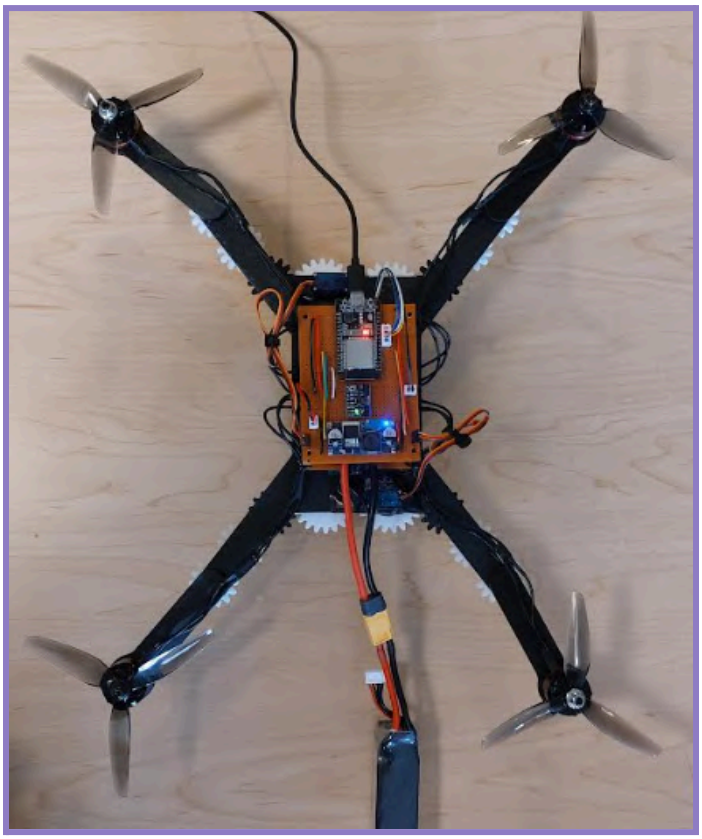
Variable Geometry Drone

Group Members: Owen Geiss, Jay Suthar, and Tabitha Kwiatkowski



MECH307

Colorado State University

05/08/26

Table of Contents

Design Summary..... 3
System Details..... 5
- Mechanical System..... 5
- Test Stand..... 8
- Electronic System..... 10
- PID (Flight Control) System..... 15

Design Evaluation..... 17
Partial Parts List..... 19
Lessons Learned..... 20
- Struggles with Electronics..... 20
- Brushless Motors and PID..... 21
- Measurement Accuracy and Design Challenges..... 22

References..... 22
Appendix..... 23
- Appendix A..... 23
- Appendix B..... 26
- Appendix C..... 28
- Appendix D..... 35

Design Summary

The variable geometry drone is an aerial device capable of altering the orientation of its secondary arms through an integrated gear and motor actuation system. transitions each secondary arm from an initial configuration to a secondary configuration using a dedicated train of spur gears mounted on either side of the frame. This works from a train of spur gears on each side of the drone. These spur gears transmit rotational motion from a servo motor to drive the controlled rotation of the secondary arms. In addition to the variable geometry mechanism, the project incorporates all fundamental components required for stable flight and control. The full drone system can be seen in ***Figure 01*** below.

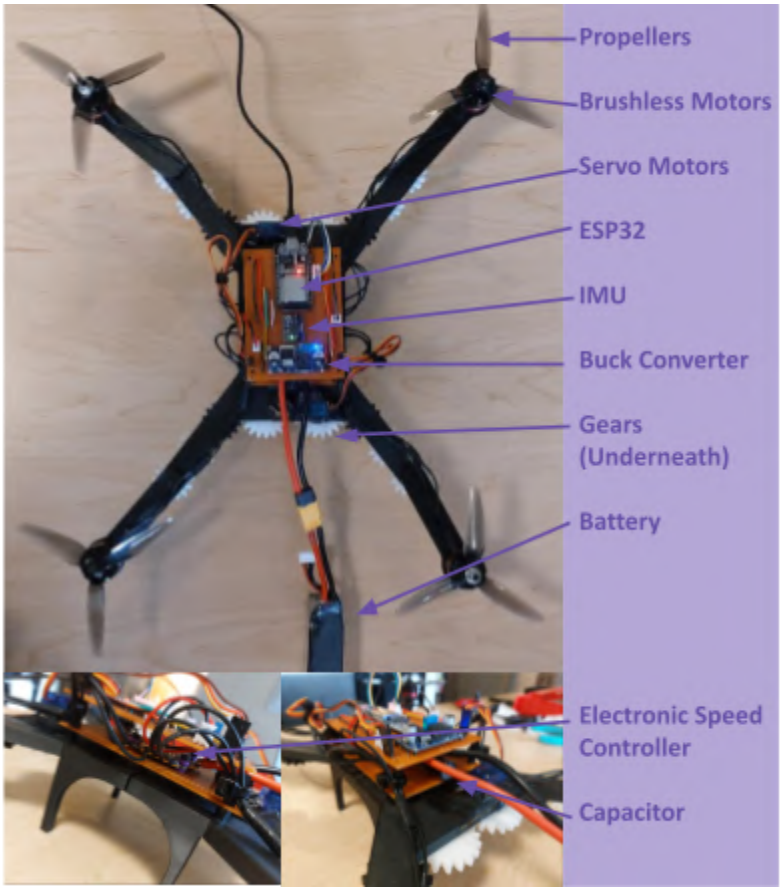


Figure 01: *Major System Components*

The electronic system of the drone provides power distribution, control, sensing, and communication required for stable flight. Electrical energy is supplied from an onboard battery which powers all major components (*motors, sensors, and flight computer*). Motor operation is managed through an electronic speed controller (ESC) which regulates power supplied to the four brushless DC motors. These motors drive propellers to generate lift and thrust for the drone.

Flight control and stabilization are controlled by an ESP-32 microcontroller. The ESP-32 functions as the flight controller interpreting user inputs and sensor data, adjusting motor speeds accordingly. Orientation and motion data are provided to the ESP-32 from an onboard inertial measurement unit (IMU). Communication and command input are transmitted via Wi-Fi and Bluetooth, while the auxiliary controllers manage operational inputs. All of the electronic components are attached to a 3D printed base.

An additional critical component was the construction of a drone test stand. The purpose of the test stand was to provide a controlled and secure environment for testing and tuning the drones proportional integral derivative (PID). The test stand was able to calibrate the pitch and roll axes while the propellers were attached. This test stand allowed for realistic testing conditions while minimizing safety risks during control system adjustments.

System Details

Mechanical System

The drone comprises three primary mechanical components: the drone base, the gear system, and the movable arms. Each component was initially hand sketched (***Figure 02***) before being modeled and refined using *Fusion 360*. Throughout the project, these components underwent multiple design iterations to accommodate changes in the overall drone architecture and to achieve appropriate mechanical tolerances [A].

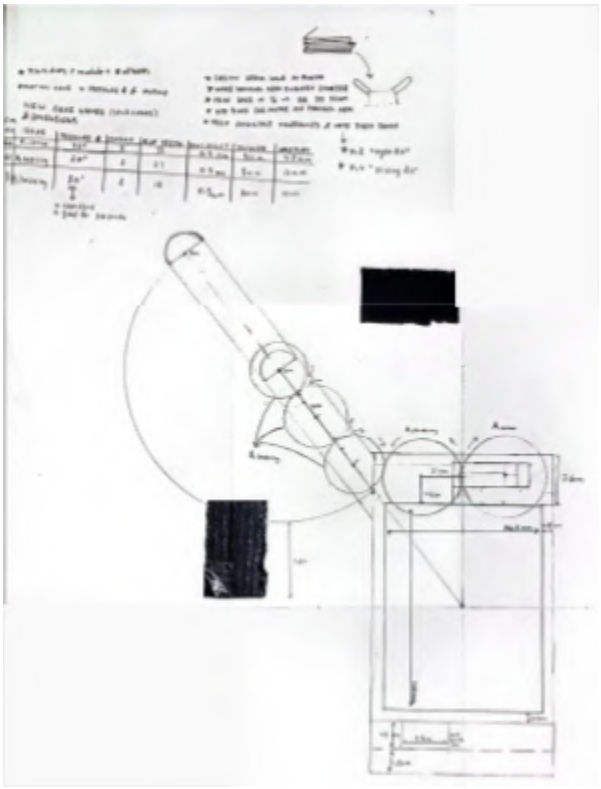


Figure 02: *Hand Sketched Design for Drone Mechanical System*

Component dimensions were designed based on the sizes of key hardware elements, including the perforated printed circuit board, battery, motors, and propellers. Once the overall dimensions of the drone base were defined, the gear sizes were selected accordingly. The gear

train was designed with a progression from larger to smaller gears to improve mechanical power transmission, enabling rapid and robust movement of the secondary arms.

An identical, mirrored gear arrangement is used on both sides of the drone, with a single servo motor driving each side. Due to this mirrored configuration, one input rotation produces synchronized and equivalent motion on both sides of the drone. This mirrored design concept was inspired by a YouTube video identified during the preliminary research phase of the project and was adapted to meet the specific requirements of the drone [1]. ***Figure 03*** and ***Table 01*** illustrate the gear layout and corresponding dimensions.

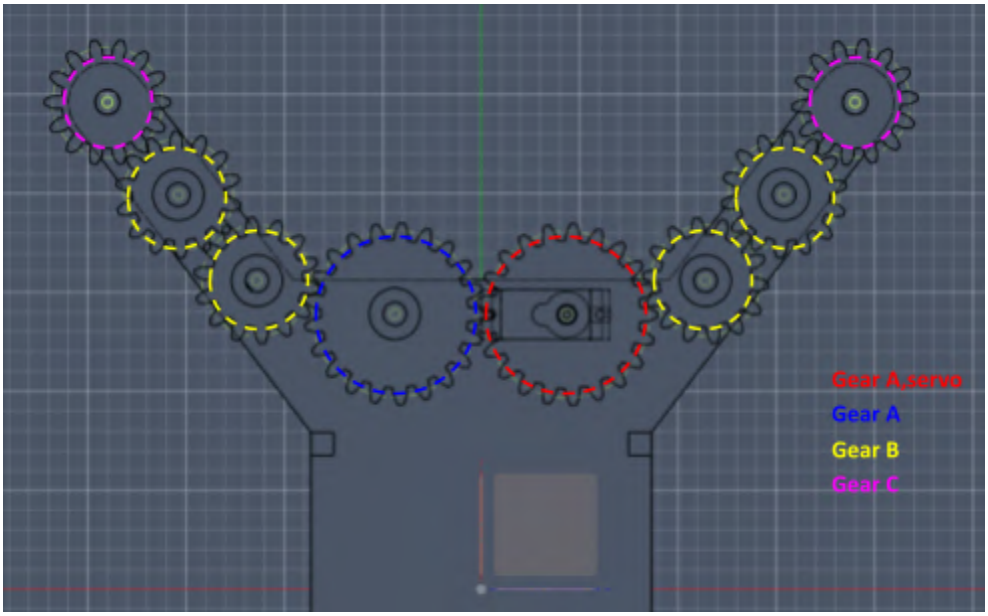


Figure 03: *Gear Layout*

For spur gears to mesh and transmit motion correctly both the pressure angle and the module must be identical for all mating gears. The module defines the tooth size and spacing while the pressure angle governs the geometry of the tooth profile. If either parameter differs between gears the teeth will not maintain contact they will fail to mesh.

Table 01: *Gear Dimensions*

Gear	Outside Diameter [mm]	Pressure Angle [degrees]	Module	# of Teeth	Root Fillet [mm]	Thickness [mm]	Inside Diameter [mm]
A,servo	4.2	20	2	21	0.5	5	5.5
A	4.2	20	2	21	0.5	5	13
B	3.0	20	2	15	0.5	5	13
C	2.8	20	2	14	0.5	5	6.2

The movable arms were dimensioned so there was sufficient clearance between propellers across intended configurations. Arm length and pivot geometry were selected based on the propeller diameter and the range of motion required. The motion of each arm is constrained through a connecting rod attached to Gear C, which converts the rotational motion of the gear train into controlled angular movement of the arm. The two drone configurations can be seen below in ***Figure 04***.

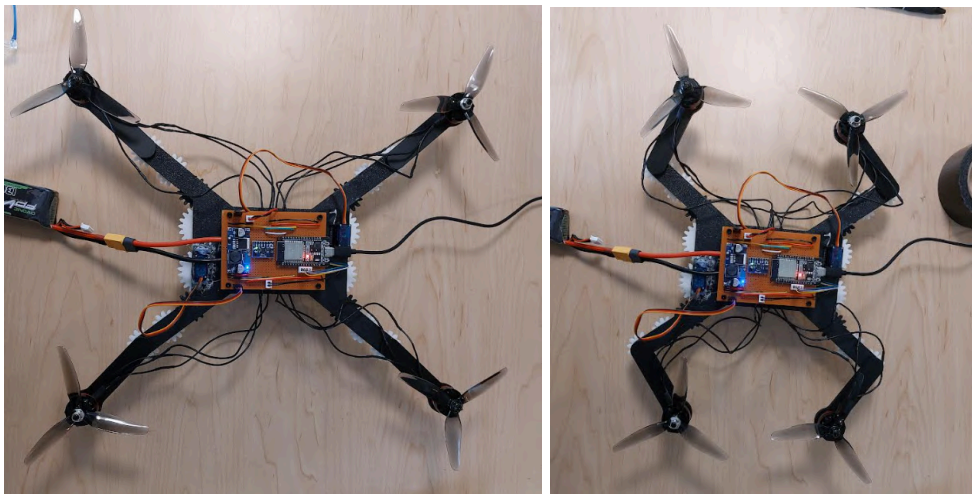


Figure 04: *Configuration One (Left) and Configuration Two (Right)*

The gear train on each side of the drone is powered by a dedicated servo motor mounted to the base. Servo motion is governed by a control code running on the microcontroller [B]. Each servo is assigned a defined initial position corresponding to the default arm configuration and a final position corresponding to the default arm configuration and final position.

Test Stand

The test stand was a critical part of designing the drone and went through multiple iterations as the project evolved and grew. The basic design philosophy of a drone test stand allows for the drone to be tested with The first iteration only had freedom of movement in 1 axis, and did not allow complete movement rotation of the drone. As shown in ***Figure 05***, it used recycled 3d printer parts, specifically the rods and bearings, with 3d prints providing structure and mounting points for the drone.

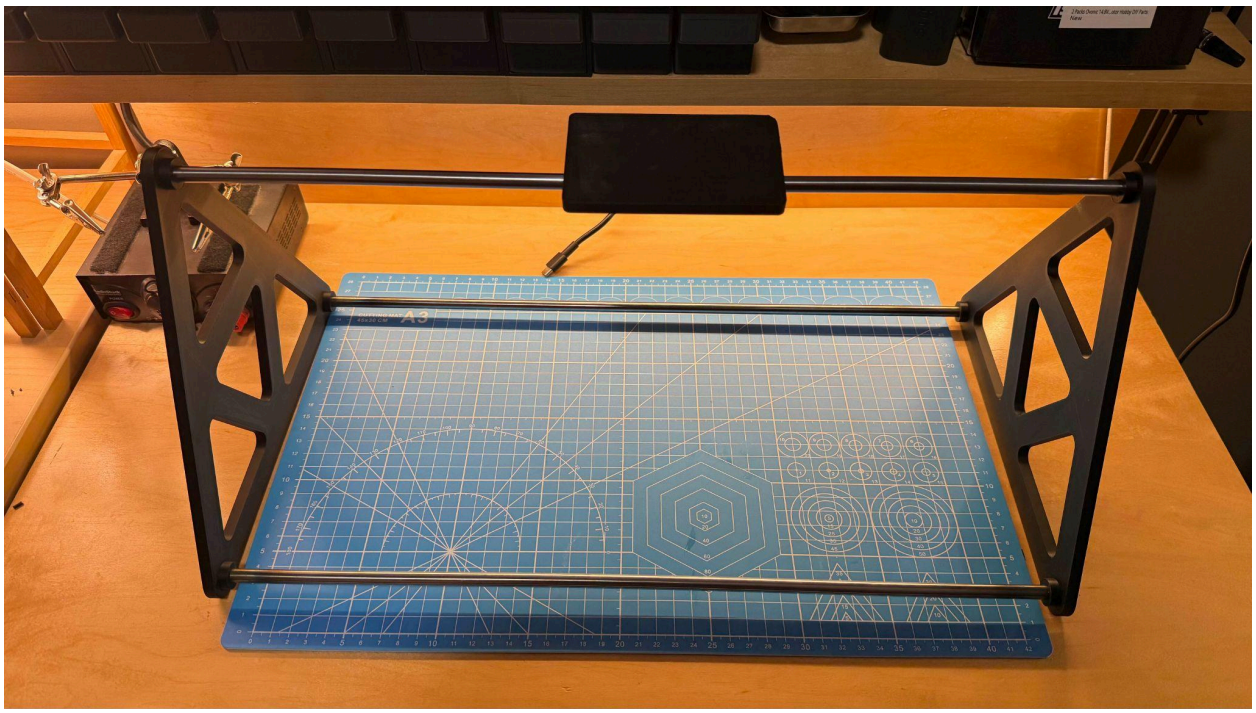


Figure 05: *Test Stand V1*

Test stand V2 then iterates on the design to allow for an additional axis of rotation, allowing the drone to rotate in Roll and Pitch. V2 utilized parts from V1 such as the drone attachment point and the steel rods, as seen in ***Figure 06***. This design allowed for full 360 deg rotation in both roll and pitch. It was decided the Yaw would not be a primary target for this project, so we did not pursue it.

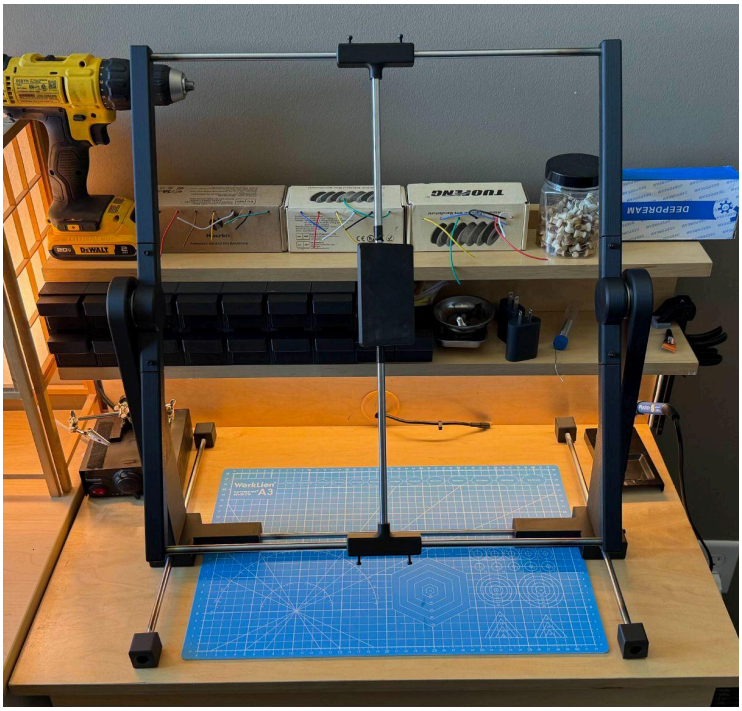


Figure 06: *Test Stand V2*

However during integration testing it was found that the propeller diameter was not fully considered in the V2 design leading to an additional design to be made, which resulted in Test Stand V3. This was the final iteration and was used during the presentation.



Figure 07: *Test Stand V2*

Electronic System

The electronic system integrates power distribution, sensing, computation, and wireless communication into a unified architecture that enables stable flight and remote operation. A complete schematic of the electronic system is shown in ***Figure 08*** below.

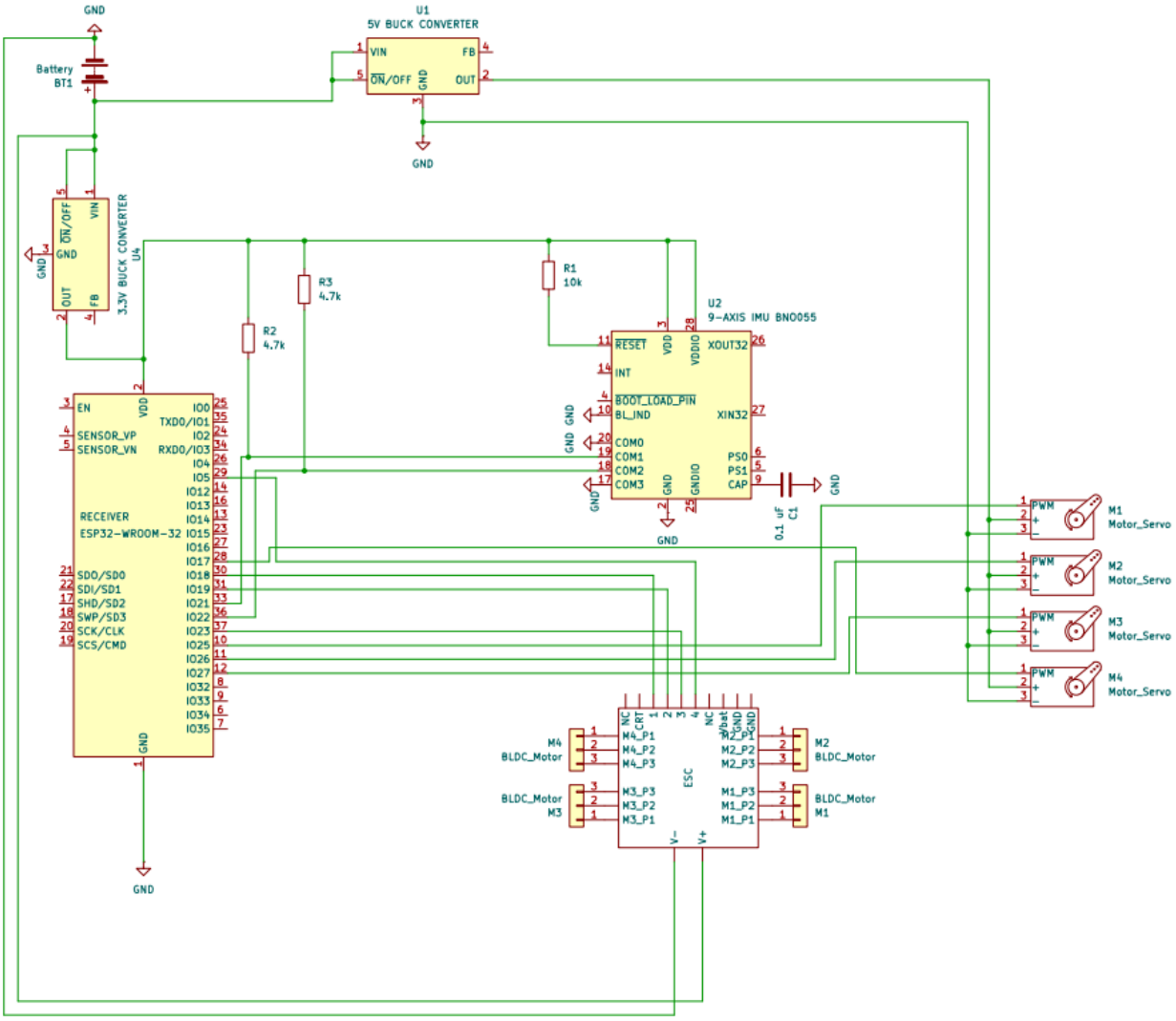


Figure 08: *Electronic System Schematic*

Power is supplied by an OVONIC 4S LiPo battery rated at 14.8V and 1300mAh with a 100C discharge rating, connected via an XT60 connector. The battery feeds two voltage regulation stages: a 5V buck converter and a 3.3V buck converter, which supply regulated power to the logic-level components. The four brushless DC motors are driven by a Radiolink FLYCOLOR 55A 4-in-1 ESC supporting 3–6S input, which receives PWM control signals directly from the flight computer and distributes three-phase power to each motor independently.

The flight computer is an ESP32-WROOM-32 microcontroller, which serves as the central processing unit for all flight control logic. It reads orientation data from an MPU-6050 inertial measurement unit over I2C (SDA on GPIO 21, SCL on GPIO 22), runs the closed-loop PID stabilization algorithm, and outputs PWM signals to the ESC on GPIO pins 25, 26, 27, and 14 for motors M1 through M4. Arm configuration is controlled by two 9g servo motors, driven on GPIO 16 (front) and GPIO 17 (rear). Wireless command reception is handled via ESP-NOW, a low-latency peer-to-peer Wi-Fi protocol that allows the drone ESP32 (MAC address 38:18:2B:B2:47:7C) to receive control packets directly from the controller ESP32 (MAC address 4C:C3:82:C4:82:A4) without a router or access point.

The operator's input device is a PlayStation 2 (PS2) gamepad interfaced with a second ESP32 microcontroller. The controller ESP32 reads joystick axes and button states via SPI using the PS2X library and packages them into a structured data packet transmitted wirelessly to the drone at approximately 50 Hz. Stick mappings include throttle on the left stick vertical axis, roll and pitch on the right stick, yaw on the left stick horizontal axis, and dedicated buttons for arm/disarm, emergency stop, and arm configuration changes.

The controller board itself is powered by a 9v battery through voltage regulator. The Ps2 controller's VCC pin uses a capacitor to help stabilize its output signals. It then has two leds to assist in ensuring network packets are not being lost. When a packet is valid and is successfully sent, the blue LED flashes. When the packet is not delivered successfully the red LED flashes. The board also has an onboard piezo electric buzzer with plays a startup tone when the board is successfully initialized.

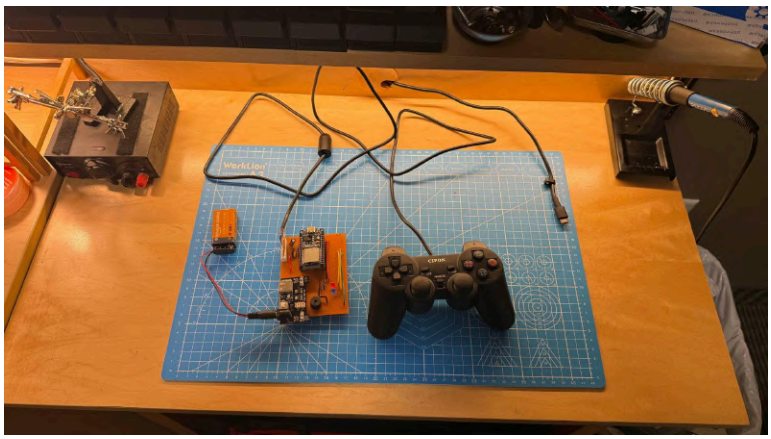


Figure 09: *Ps2 Controller and Interface Board*

The PS2 controller was chosen as it has good ergonomics and they are very easy to come by, making it a perfect choice for this project. An additional benefit of this platform is because libraries have been written to interpret it for a long time. For this we used PsxLib, which was originally written in 2007 and was modernized by Giuseppe Porcheddu. This library allowed for simplified interpretation of the PS2 controllers binary output. The software itself can be found in Appendix C and D, with the flowcharts being found below in Figures 11 and 12. Most calculations were done on the Drone, with the controller only passing along PS2 controller signals to the Drone. This allowed for the PID to be run completely locally on the drone.

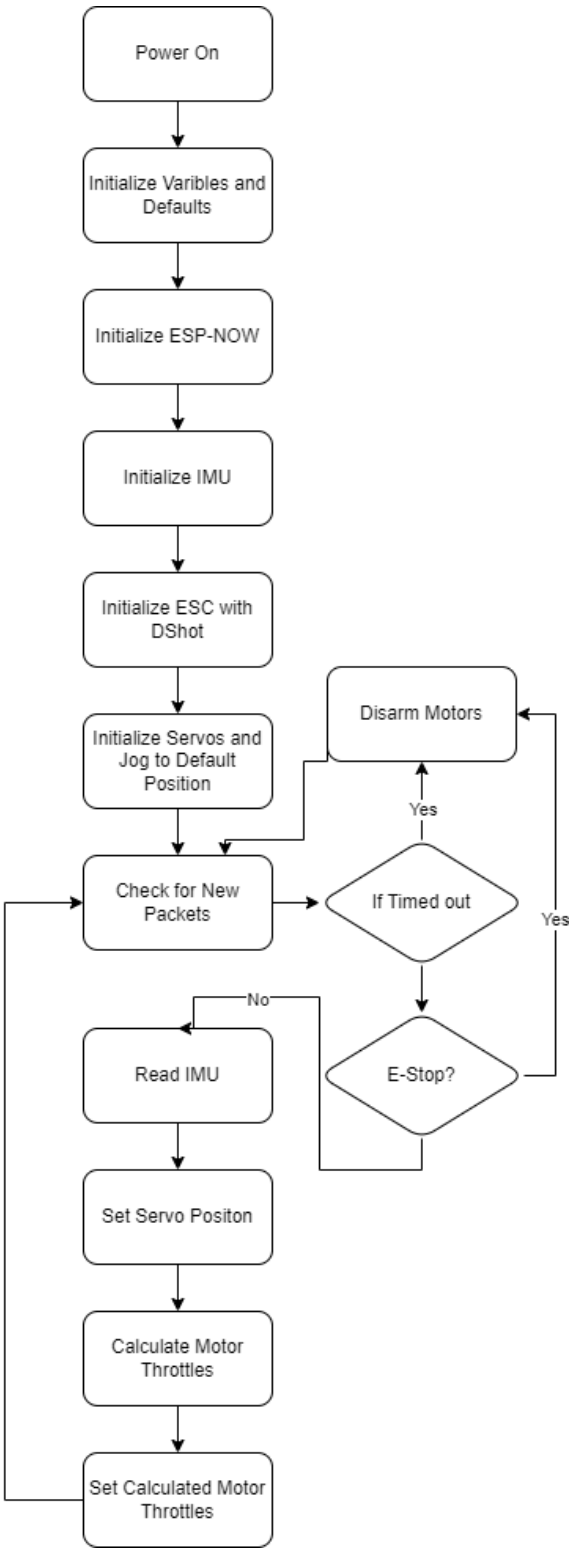


Figure 10: *Drone Software Flowchart*

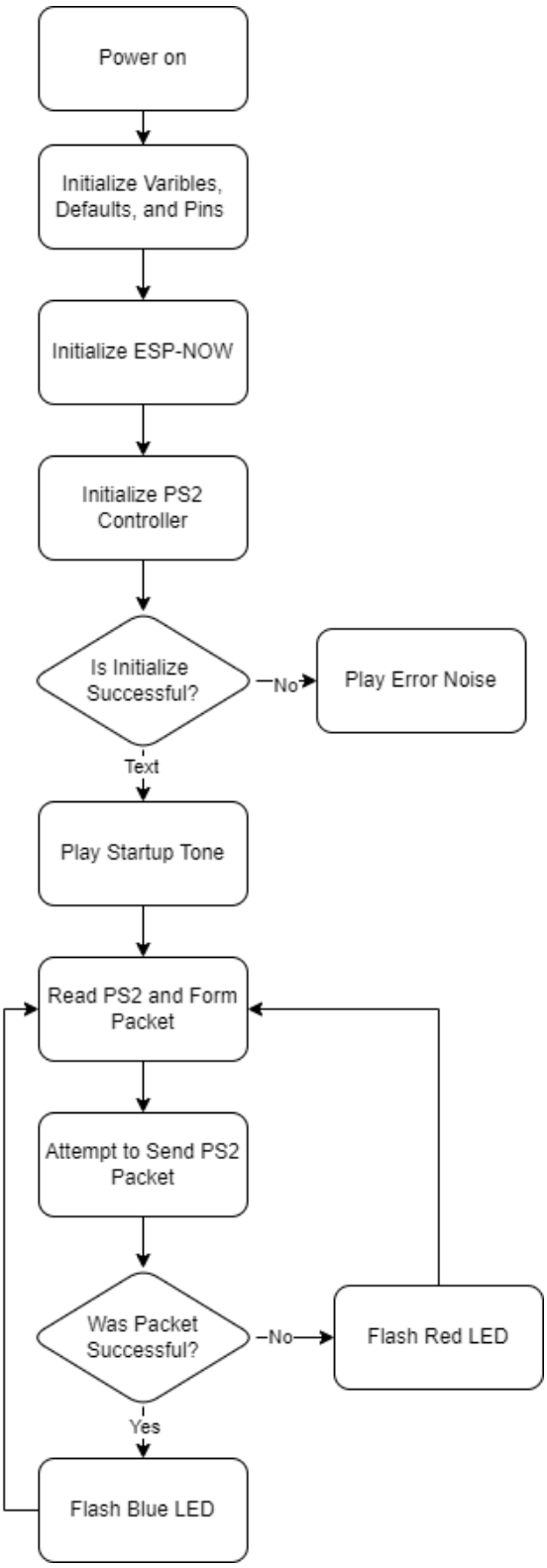


Figure 11: *Controller Software Flowchart*

PID (Flight Control) System

The drone uses a closed-loop Proportional-Integral-Derivative (PID) control system to maintain stable flight. The MPU-6050 IMU continuously measures angular orientation and angular rates across three axes: roll, pitch, and yaw. These measurements serve as the feedback signal for three independent PID loops, one per axis, which run on the ESP32 flight computer at each control cycle.

The general discrete-time PID equation implemented in firmware is:

$$u[k] = Kp \cdot e[k] + Ki \cdot \Sigma e[k] \cdot dt + Kd \cdot (e[k] - e[k-1]) / dt$$

where e[k] is the error between the desired setpoint and the measured angle at timestep k, Kp is the proportional gain, Ki is the integral gain, Kd is the derivative gain, and dt is the time between control cycles. The proportional term corrects for the current error magnitude, the integral term eliminates steady-state offset by accumulating error over time, and the derivative term damps oscillation by responding to the rate of error change.

The three PID output signals are combined in a mixer that converts desired roll, pitch, yaw, and throttle commands into individual motor speed commands for the four brushless motors. A digital low-pass filter (DLPF) was applied to the MPU-6050 at a 44 Hz bandwidth to attenuate high-frequency vibration noise from the spinning motors before the sensor data reached the PID loops. IMU orientation was estimated using a complementary filter that blends accelerometer and gyroscope data. This approach corrects roll and pitch drift using gravity as a reference while being computationally lightweight enough to run in real time on the ESP32.

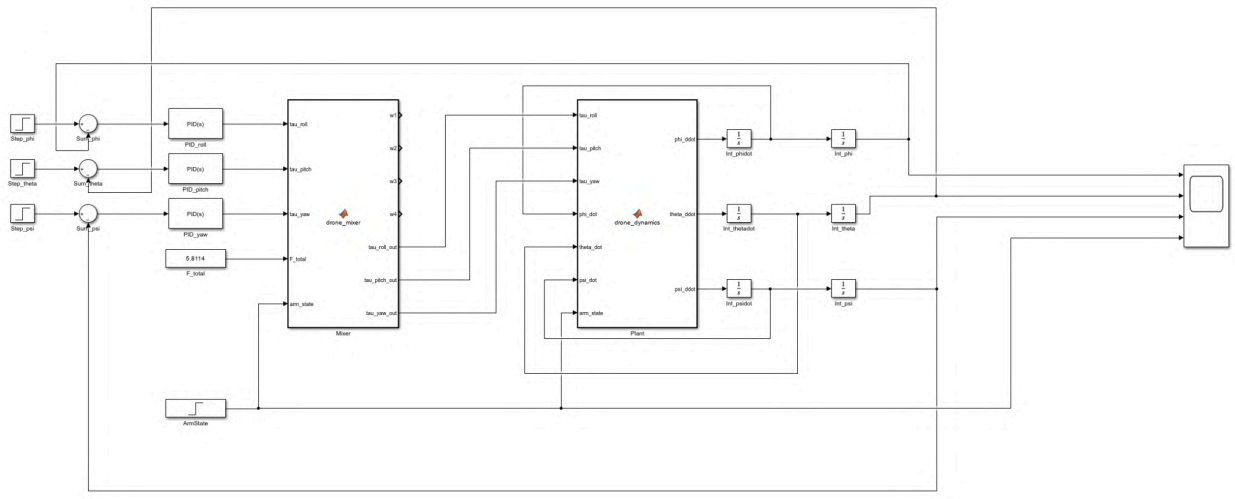


Figure 12: *MATLAB Simulink PID Simulation of a Fixed-Arm Drone with 3 preset inputs in each individual axis: pitch, roll, and yaw.*

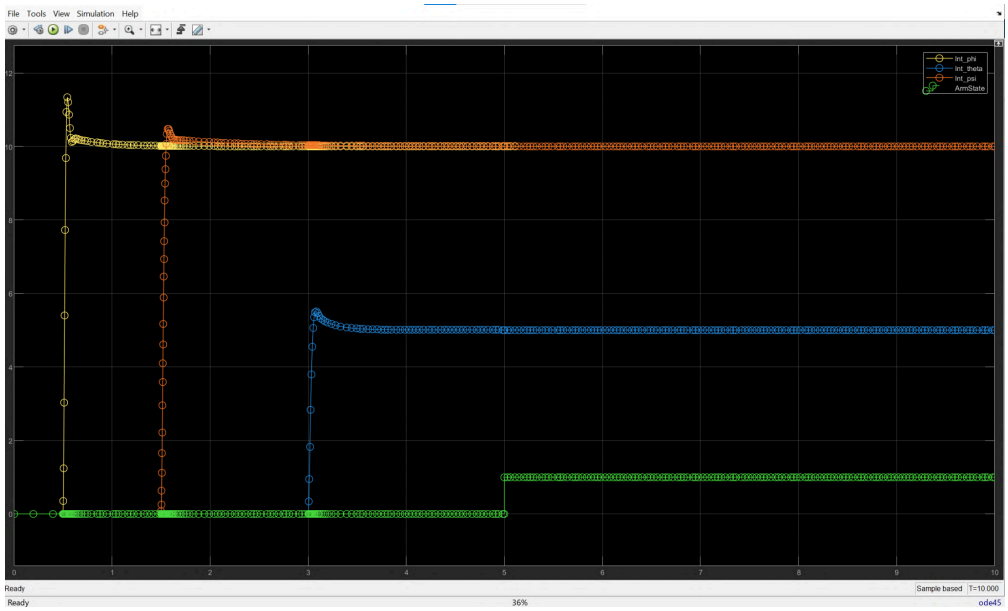


Figure 13: *Simulation output demonstrating PID gains correcting disturbances. Each line showcases a separate axis, with the green line indicating a change in the Moment of Inertia due to the arms of the drone folding in.*

Design Evaluation

The variable geometry drone was evaluated on a multi-axis test stand that constrained the vehicle's position while allowing rotation about in 2 axes. Under these conditions, the closed-loop PID flight control system demonstrated the ability to self-stabilize the drone for approximately one second before accumulated error caused the system to diverge. This represents a partial success for the stabilization system: the controller was able to generate corrective motor commands in response to IMU feedback, but the PID gains were not fully tuned to the point of sustained stable hover.

The variable geometry mechanism was not functional at the time of the final demonstration. The 9g servo motors selected to actuate the gear train were insufficient to overcome the torque loading caused by the motor vibration and gravitational forces acting on the extended arms. This caused the secondary arms to deflect under load rather than rotate as designed. To enable the drone to be demonstrated with a stable platform, the arms were fixed in place with adhesive for the final evaluation. The gear train and mechanical linkage design were validated in static testing and functioned correctly when actuated by hand, confirming that the core mechanism concept is sound but requires higher-torque actuators for operational use.

The electronic and communication systems performed as intended throughout testing. The ESP-NOW wireless link between the controller and drone ESP32 maintained reliable low-latency communication, and all manual control inputs (throttle, arm/disarm, and emergency stop) were correctly received and acted upon. The MPU-6050 IMU provided consistent orientation data, and the onboard digital low-pass filtering effectively reduced motor

vibration noise in the sensor readings. The overall system architecture, including the power distribution chain, ESC operation, and microcontroller integration, functioned correctly and provided a solid foundation for further flight control tuning.

Table 02: *Project functionality requirements vs. Project-specific parts list*

Output Display	Packet Success/Failure LEDs
Audio Output Device	Controller Board Startup Noise ESC Startup Noise
Manual User Input	PS2 Controller
Automatic Sensor	Accelerometer Gyroscope
Actuators, Mechanisms, and Hardware	Servo Motors DC Brushless Motors Arm Gearing
Microcontrollers	2X ESP32's
Logic, Processing, and Control	PID Control Loop

Partial Parts List

For all major components of the drone the part name, description, model number, source, and price are listed below in ***Table 03***.

Table 03: *Partial Parts List*

Part Name	Description	Model Number	Source	Price
ESP32	Microcontroller with built in Wi-Fi and Bluetooth used as the main control unit for processing inputs and running control logic.	ESP-32S Devkit V1	Amazon	$6.67
Drone Motors	Motors designed to provide high thrust and efficiency to generate lift and propulsion.	AKK RS2205	Amazon	$33.99
ESC	Electronic speed controller that regulates the speed and direction of the brushless motors.	Flycolor Raport Bls-04	Amazon	$49.99
Servos	Used for precise angular control of mechanical components.	SG90	Amazon	$1.99
PS2 Controller	Used as a human interface device to provide inputs for remote control of the drone.	PS2	Amazon	$13.99
2.54 mm Connectors	Used to connect the ESC and power to the drone control board	JST-XH	Amazon	$12.99
1 mm Connectors	Used to connect the ESC without damaging the board.	JST-SH	Amazon	$16.99
2.54 mm Pin Connectors	Used to connect to the Servo Motors without extensive modification.	Dupont	Amazon	$12.99
LiPo Battery	Powered the drones systems and motors.	4s 100C 1300mAh 14.8V	Amazon	$36.99

Lessons Learned

Struggles with Electronics

The difficulties in building a drone cannot be understated. While a seemingly simple endeavor since quad-copter drones have become commonplace, it was not as such. Electronics in this capacity become difficult to deal with. A big roadblock was learning the best way to connect different connection standards together. The Flycolor ESC's amazon page did not list what the connector type was. This was from their assumption that the consumer would just buy a stock Flight Computer. This lead to a multi week effort to identify and correctly purchase the connector type that the ESC used for its data. This ended up being a 10 pin JST-SH male connector which follows a 1 mm pitch unlike most applications which use a 2.54 mm (.1 in) pitch. This was then wired exactly as the provided connector wire was. This became an issue later on as the Vbat line was left unconnected. The Vbat line was a direct connection to the battery with the purpose being able to measure the voltage of the battery. This wire was not supposed to carry current so it was not used to power the ESP32 and IMU. After a test the Vbat connection shifted to a point where it connected to the 3.3V pin on the ESP32, which proceed drive 14.8V to 16.8V into the ESP32 and IMU, rendering both inoperable. From this, we learned the importance of wire management as well as the difficulty with working with systems that do not have proper documentation. After this mistake we immediately worked to mitigate this problem from occurring in the future.

A lesson learned not from this project but from MECH 202, was the importance of electronic mounting. This was an opportunity to improve upon skills learned in that class. For

this a proto board was necessary. To make the connections more reliable we used copper strip board, which allowed for easier connections between components. For students in future MECH 307 courses, it would be beneficial for everyone to receive these to help in the final design wiring.

Brushless Motors and PID

Getting the motors running cleanly took longer than expected. The ESP32Servo library works fine for servos but is the wrong tool for brushless motors. After some research we switched to a DSHOT protocol approach, which required starting over on that part of the code. IMU calibration was similarly tedious. Getting the axis orientation right and making sure the sensor readings were clean and consistent was very time consuming. On the PID side, the integral term made the drone more snappy than effective at our level of tuning, so we dropped it and utilized just P and D gains. That gave us roughly one second of self-stabilization we showed at the demo. The edit-compile-upload-test loop was slow. Every small firmware change led us to re-upload and retest, which added up fast. If we were starting over, we would begin earlier, research how similar drone builds actually played out, and go into testing with more realistic expectations for what we could get working in the time we had.

Measurement Accuracy and Design Challenges

Ensuring the use of accurate measurement devices is critical during the design and prototyping phases. For this project, new digital calipers were purchased to improve the precision of material measurements. The introduction of the digital calipers significantly reduced the design iteration time which had previously been prolonged due to inaccurate

measurements and repeated prototype revisions. Improved measurement accuracy allowed components to fit more reliably and reduced the number of failed design iterations.

Additionally, challengers were encountered with gear meshing and power transmission due to the use of 3D printed gears. Several issues arose, including loose tolerances and gear slippage under load. If the mechanical system of the drone were to be redesigned greater emphasis would be placed on using high quality measurement tools earlier and utilizing higher quality resolution 3D printers earlier. Incorporating kinematic and load simulations within the modeling software prior to printing would have been beneficial in identifying gear engagement issues.

References

[1] Micheal Rechtin. (2024, November 18). *Building a Real Life Transformer* [Video]. YouTube. https://www.youtube.com/watch?v=f1GSzysrYtw

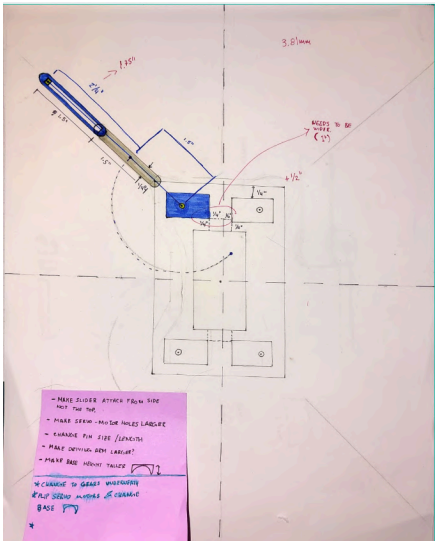
[2] Wastl Kraus. (2025, November). *DShotRMT - ESP32 RMT DShot Driver. Github.* https://github.com/derdoktor667/DShotRMT

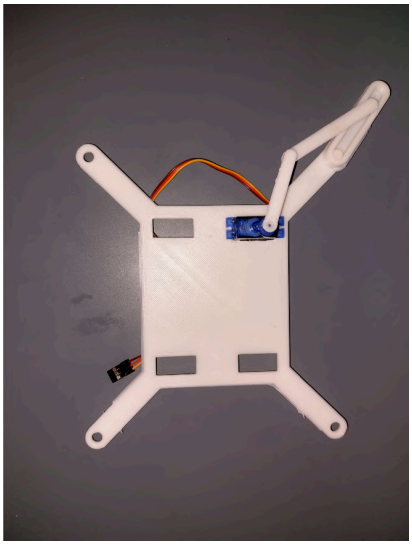
[3] Giuseppe Porcheddu. (2019, April). PsxLib - ESP32 Ps2 Library. Github. https://github.com/GiuseppePorcheddu/PsxLib

Appendix

Appendix A

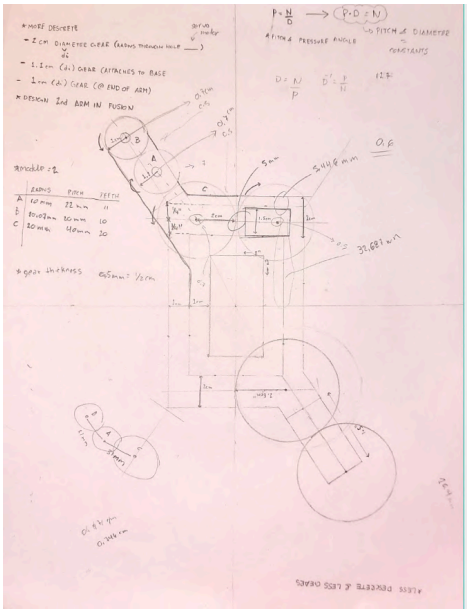
Prototype (1)





Issues: Crank / Slider method would ‘lock up’ too much and was too complicated.

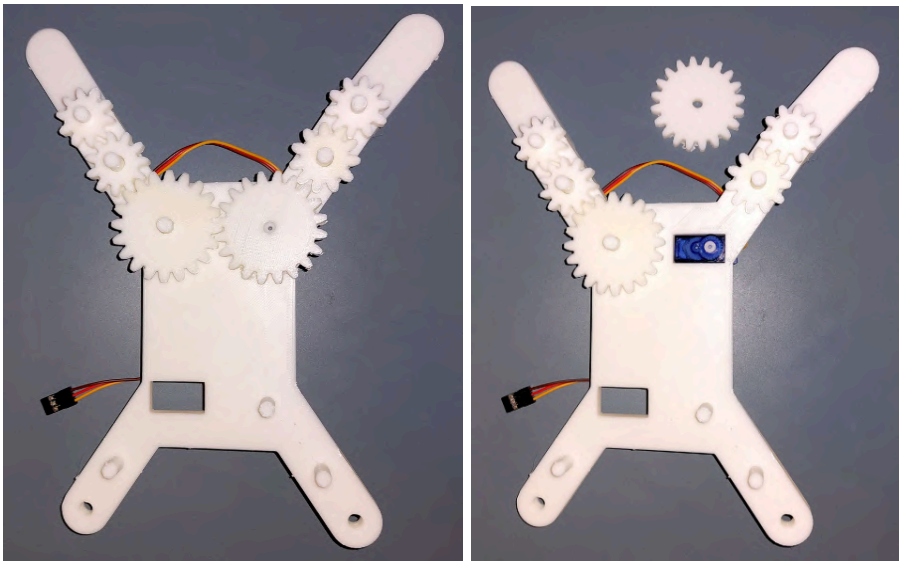
Prototype (2)





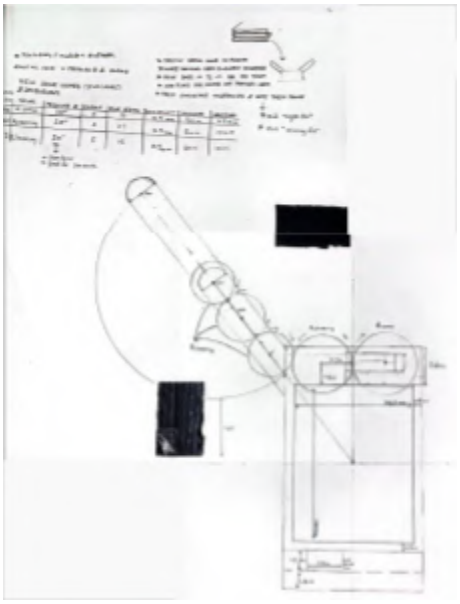
Issues: Base was too thick and the gears were a little too close together. This prototype was mainly a test to see how well the gears fit together.

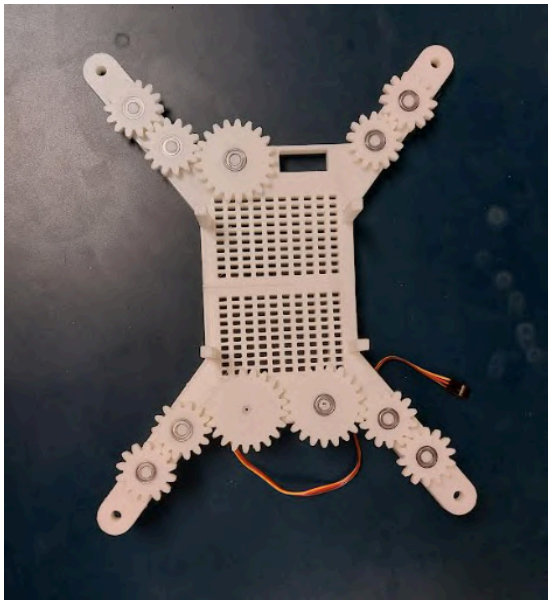
Prototype (4)



Issues: The gears match up well for this prototype, the issues left to fix are the tolerance on the moving arm (currently too tight of a fit for the servo motor to move gears) and possibly adjusting the servo motor side gears distances.

Prototype (5)





Issues: The center of the drone needs to be changed for more support and longer legs for proper clearance of the battery.

Prototype (6)



This was the final design with all of the gears assembled (No extended arms or hardware added).

Appendix B

```
#include <ESP32Servo.h>

// Create servo objects
Servo servo0;
Servo servo1;

// ESP32 PWM-capable pins
int servo0Pin = 19;
int servo1Pin = 32;

// Motion tuning
int steps = 50;         // Higher = smoother
int stepDelay = 20;     // ms between steps

// Position storage
int servo0Start, servo0End;
int servo1Start, servo1End;

// 10 70 90 160

bool positionsSet = false;

void setup() {
  Serial.begin(9600);

  // Set PWM period for ESP32 servos
  servo0.setPeriodHertz(50);
  servo1.setPeriodHertz(50);

  // Attach servos (min/max pulse width optional but recommended)
  servo0.attach(servo0Pin, 500, 2400);
  servo1.attach(servo1Pin, 500, 2400);
```

```
  Serial.println("Enter: servo0_start servo0_end servo1_start
servo1_end");
}

void moveServos(int s0Start, int s0End, int s1Start, int s1End) {
  float servo0Step = (s0End - s0Start) / (float)steps;
  float servo1Step = (s1End - s1Start) / (float)steps;

  for (int i = 0; i <= steps; i++) {
    servo0.write(s0Start + servo0Step * i);
    servo1.write(s1Start + servo1Step * i);
    delay(stepDelay);
  }
}

void loop() {

  // Read initial/final positions
  if (!positionsSet && Serial.available() >= 4) {
    servo0Start = Serial.parseInt();
    servo0End   = Serial.parseInt();
    servo1Start = Serial.parseInt();
    servo1End   = Serial.parseInt();

    // Move to initial positions
    servo0.write(servo0Start);
    servo1.write(servo1Start);

    positionsSet = true;

    Serial.println("Positions saved.");
    Serial.println("Enter F (forward) or B (backward).");
  }

  // Read movement commands
  if (positionsSet && Serial.available() > 0) {
    char command = Serial.read();

    if (command == 'F' || command == 'f') {
      Serial.println("Moving Initial → Final");
```

```
      moveServos(servo0Start, servo0End, servo1Start, servo1End);
    }

    if (command == 'B' || command == 'b') {
      Serial.println("Moving Final → Initial");
      moveServos(servo0End, servo0Start, servo1End, servo1Start);
    }
  }
}
```

Appendix C

```
#include "PsxLib.h"
#include <esp_now.h>
#include <WiFi.h>


#define PS2_DAT   19
#define PS2_CMD   23
#define PS2_CLK   18
#define PS2_ATT    5
#define PS2_DELAY 100


#define BUZZER_PIN   25
#define LED_OK_PIN   27
#define LED_FAIL_PIN 26
#define LED_FLASH_MS 20


uint8_t droneMac[] = {0x38, 0x18, 0x2B, 0xB2, 0x47, 0x7C};


typedef struct __attribute__((packed)) ControlPacket {
  uint16_t buttons;
  uint32_t seq;
} ControlPacket;
```

```
Psx          ps2;
ControlPacket pkt;
uint32_t seq      = 0;
bool ps2OK    = false;
uint32_t sendOK   = 0;
uint32_t sendFail = 0;


unsigned long ledOkOffAt   = 0;
unsigned long ledFailOffAt = 0;

void onSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {
  if (status == ESP_NOW_SEND_SUCCESS) {
    sendOK++;
    digitalWrite(LED_OK_PIN, HIGH);
    ledOkOffAt = millis() + LED_FLASH_MS;
  } else {
    sendFail++;
    digitalWrite(LED_FAIL_PIN, HIGH);
    ledFailOffAt = millis() + LED_FLASH_MS;
  }
}


void initPS2() {
  Serial.println("--- Initialising PS2 controller ---");

  ps2.setupPins(PS2_DAT, PS2_CMD, PS2_ATT, PS2_CLK, PS2_DELAY);

  unsigned int result = ps2.read();
  if (result != 0xFFFF) {
    ps2OK = true;
    Serial.println("  PS2 OK!");
  } else {
    Serial.println("  PS2 FAILED — check wiring:");
    Serial.println("    DAT→19  CMD→23  CLK→18  ATT→5");
    Serial.println("    VCC→3.3V  GND→GND");
    Serial.println("    10kΩ pull-up on DAT and CMD");
    ps2OK = false;
  }
```

```
}


void playStartupTune(bool success) {
  if (success) {
    tone(BUZZER_PIN, 880, 100);
    delay(120);
    tone(BUZZER_PIN, 1175, 100);
    delay(120);
    tone(BUZZER_PIN, 1568, 150);
    delay(170);
  }
  else {
    tone(BUZZER_PIN, 392, 200);
    delay(220);
    tone(BUZZER_PIN, 262, 400);
    delay(420);
  }
  noTone(BUZZER_PIN);
}

void setup() {
  Serial.begin(115200);
  delay(500);

  pinMode(BUZZER_PIN,   OUTPUT);
  pinMode(LED_OK_PIN,   OUTPUT);
  pinMode(LED_FAIL_PIN, OUTPUT);
  digitalWrite(LED_OK_PIN,   LOW);
  digitalWrite(LED_FAIL_PIN, LOW);


  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  Serial.printf("Controller MAC: %s\n", WiFi.macAddress().c_str());

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init FAILED — restarting");
    delay(2000);
    ESP.restart();
```

```
  }
  Serial.println("ESP-NOW init OK");
  esp_now_register_send_cb(onSent);

  esp_now_peer_info_t peer = {};
  memcpy(peer.peer_addr, droneMac, 6);
  peer.channel = 1;
  peer.encrypt = false;
  esp_now_add_peer(&peer);
  Serial.printf("%02X:%02X:%02X:%02X:%02X:%02X\n", droneMac[0],
droneMac[1], droneMac[2], droneMac[3], droneMac[4], droneMac[5]);

  delay(300);
  initPS2();

  playStartupTune(ps2OK);
}

void loop() {
  unsigned long now = millis();
  if (ledOkOffAt   && now >= ledOkOffAt)   {
    digitalWrite(LED_OK_PIN,   LOW); ledOkOffAt   = 0;
  }
  if (ledFailOffAt && now >= ledFailOffAt) {
    digitalWrite(LED_FAIL_PIN, LOW); ledFailOffAt = 0;
  }


  if (!ps2OK) {
    Serial.println("PS2 not connected — retrying...");
    initPS2();
    delay(2000);
    return;
  }


  ps2.read();

  pkt.buttons = 0;
```

```
if (ps2.button(PSB_SELECT)){
  pkt.buttons |= PSB_SELECT;
}
if (ps2.button(PSB_START)){
  pkt.buttons |= PSB_START;
}
if (ps2.button(PSB_PAD_UP)){
  pkt.buttons |= PSB_PAD_UP;
}
if (ps2.button(PSB_PAD_RIGHT)){
  pkt.buttons |= PSB_PAD_RIGHT;
}
if (ps2.button(PSB_PAD_DOWN)){
  pkt.buttons |= PSB_PAD_DOWN;
}
if (ps2.button(PSB_PAD_LEFT)){
  pkt.buttons |= PSB_PAD_LEFT;
}
if (ps2.button(PSB_L2)){
  pkt.buttons |= PSB_L2;
}
if (ps2.button(PSB_R2)){
  pkt.buttons |= PSB_R2;
}
if (ps2.button(PSB_L1)){
  pkt.buttons |= PSB_L1;
}
if (ps2.button(PSB_R1)){
  pkt.buttons |= PSB_R1;
}
if (ps2.button(PSB_TRIANGLE)){
  pkt.buttons |= PSB_TRIANGLE;
}
if (ps2.button(PSB_CIRCLE)){
  pkt.buttons |= PSB_CIRCLE;
}
if (ps2.button(PSB_CROSS)){
  pkt.buttons |= PSB_CROSS;
}
if (ps2.button(PSB_SQUARE)){
```

```
  pkt.buttons |= PSB_SQUARE;
}

pkt.seq = seq++;


esp_now_send(droneMac, (uint8_t *)&pkt, sizeof(pkt));

// ── Serial debug every 200ms ──────────────────────────
static unsigned long lastPrint = 0;
if (millis() - lastPrint >= 200) {
  lastPrint = millis();

  Serial.printf("BTN:0x%04X  seq:%lu  ok:%lu  fail:%lu",
                pkt.buttons, pkt.seq, sendOK, sendFail);

  if (pkt.buttons) {
    Serial.print("  [");
    if (pkt.buttons & PSB_SELECT){
      Serial.print(" SEL");
    }
    if (pkt.buttons & PSB_START){
      Serial.print(" START");
    }
    if (pkt.buttons & PSB_PAD_UP){
      Serial.print(" UP");
    }
    if (pkt.buttons & PSB_PAD_DOWN){
      Serial.print(" DOWN");
    }
    if (pkt.buttons & PSB_PAD_LEFT){
      Serial.print(" LEFT");
    }
    if (pkt.buttons & PSB_PAD_RIGHT){
      Serial.print(" RIGHT");
    }
    if (pkt.buttons & PSB_L1){
      Serial.print(" L1");
    }
    if (pkt.buttons & PSB_R1){
```

```
        Serial.print(" R1");
      }
      if (pkt.buttons & PSB_L2){
        Serial.print(" L2");
      }
      if (pkt.buttons & PSB_R2){
        Serial.print(" R2");
      }
      if (pkt.buttons & PSB_TRIANGLE){
        Serial.print(" TRI");
      }
      if (pkt.buttons & PSB_CIRCLE){
        Serial.print(" CIRC");
      }
      if (pkt.buttons & PSB_CROSS){
        Serial.print(" CROSS");
      }
      if (pkt.buttons & PSB_SQUARE){
        Serial.print(" SQR");
      }
      Serial.print(" ]");
    }
    Serial.println();
  }

  delay(10);
}
```

Appendix D

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_MPU6050.h>
#include "DShotRMT.h"
#include "driver/ledc.h"
#include <esp_now.h>
#include <WiFi.h>
#include "esp_wifi.h"
```

```
#define MOTOR_FL_PIN  27
#define MOTOR_FR_PIN  25
#define MOTOR_RR_PIN  26
#define MOTOR_RL_PIN  14
#define BASE_THROTTLE  0.15f
#define MAX_THROTTLE  0.50f


#define SERVO_FRONT_PIN  16
#define SERVO_REAR_PIN   17


#define SERVO_FRONT_CH   0
#define SERVO_REAR_CH    1




#define SERVO_FREQ_HZ        50
#define SERVO_MIN_US         500
#define SERVO_MAX_US         2400

#define SERVO_FRONT_EXTENDED  10
#define SERVO_FRONT_CLOSED    70
#define SERVO_REAR_EXTENDED   90
#define SERVO_REAR_CLOSED     160
#define SERVO_DEFAULT_FRONT   10
#define SERVO_DEFAULT_REAR    90


#define DSHOT_MIN_PCT   0.0f
#define DSHOT_MAX_PCT   100.0f


#define LOOP_HZ    250
#define LOOP_US    (1000000 / LOOP_HZ)


struct PIDGains { float Kp, Ki, Kd; };
```

```
//                          Kp      Ki      Kd
PIDGains rollGains  = {  3.00f,  0.0f,  0.50f };
PIDGains pitchGains = {  2.42f,  0.0f,  0.50f };
PIDGains yawGains   = {  1.50f,  0.0f,  0.60f };

#define ITERM_LIMIT    50.0f
#define MAX_ANGLE_DEG  30.0f
#define MAX_YAW_RATE   90.0f
#define MIXER_SCALE    400.0f

#define THROTTLE_STEP  0.002f


#define COMM_TIMEOUT_MS  500



Adafruit_MPU6050 mpu;


DShotRMT motorFL(GPIO_NUM_27, DSHOT300);
DShotRMT motorFR(GPIO_NUM_25, DSHOT300);
DShotRMT motorRR(GPIO_NUM_26, DSHOT300);
DShotRMT motorRL(GPIO_NUM_14, DSHOT300);



typedef struct __attribute__((packed)) ControlPacket {
  uint16_t buttons;
  uint32_t seq;
} ControlPacket;


#define BTN_SELECT   0x0001
#define BTN_START    0x0008
#define BTN_L2       0x0100
#define BTN_R2       0x0200
#define BTN_L1       0x0400
#define BTN_R1       0x0800
```

```
#define BTN_TRI      0x1000
#define BTN_CIRC     0x2000
#define BTN_CROSS    0x4000
#define BTN_SQR      0x8000
#define BTN_UP       0x0010
#define BTN_RIGHT    0x0020
#define BTN_DOWN     0x0040
#define BTN_LEFT     0x0080


volatile ControlPacket lastPkt;
volatile bool          gotNew   = false;


uint32_t      pktCount   = 0;
uint32_t      dropCount  = 0;
uint32_t      lastSeq    = 0;
bool          firstPkt   = true;
uint32_t      pktThisSec = 0;
unsigned long lastStatMs = 0;
unsigned long lastPktMs  = 0;



struct ControllerInput {
  float throttle;
  float roll;
  float pitch;
  float yaw;
  float armAngle;
  bool  armed;
};

ControllerInput ctrl     = {0.0f, 0.0f, 0.0f, 0.0f, 1.0f, false};
bool            startPrev = false;



struct PIDState { float integral; };
```

```
PIDState rollState  = {0};
PIDState pitchState = {0};
PIDState yawState   = {0};



struct IMUData {
  float roll, pitch, yaw;
  float rollRate, pitchRate, yawRate;
};
IMUData imu = {0};



void  initESPNow();
void  onRecv(const esp_now_recv_info_t *info, const uint8_t *data, int
len);
void  consumePacket();
void  translatePacket(const ControlPacket &pkt);
void  armESCs();
void  readIMU(float dt);
float computePID(float sp, float meas, float rate,
                 PIDGains &g, PIDState &s, float dt);
void  mixAndWriteMotors(float thr, float r, float p, float y);
void  writeDShot(DShotRMT &motor, float pct);
void  sendAllDShot(float pct);
void  updateServos();
void  servoWrite(int channel, int pin, int deg);
float clampf(float v, float lo, float hi);
void  safeStop();
void  printLinkStats();



void setup() {
  Serial.begin(115200);
  delay(500);


  initESPNow();
```

```
if (!mpu.begin()) {
  Serial.println("MPU6050 not found — check wiring.");
  while (1);
}
mpu.setAccelerometerRange(MPU6050_RANGE_4_G);
mpu.setGyroRange(MPU6050_RANGE_500_DEG);
mpu.setFilterBandwidth(MPU6050_BAND_44_HZ);
delay(500);
Serial.println("MPU6050 ready.");


motorFL.begin();
motorFR.begin();
motorRR.begin();
motorRL.begin();


Serial.println("Initialising DShot signal...");
unsigned long t = millis();
while (millis() - t < 2000) {
  sendAllDShot(0);
  delay(1);
}

ledc_timer_config_t timer = {};
timer.speed_mode      = LEDC_LOW_SPEED_MODE;
timer.timer_num       = LEDC_TIMER_0;
timer.duty_resolution = LEDC_TIMER_16_BIT;
timer.freq_hz         = SERVO_FREQ_HZ;
timer.clk_cfg         = LEDC_AUTO_CLK;
ledc_timer_config(&timer);

ledc_channel_config_t chFront = {};
chFront.gpio_num   = SERVO_FRONT_PIN;
chFront.speed_mode = LEDC_LOW_SPEED_MODE;
chFront.channel    = (ledc_channel_t)SERVO_FRONT_CH;
chFront.timer_sel  = LEDC_TIMER_0;
chFront.duty       = 0;
```

```
  chFront.hpoint     = 0;
  ledc_channel_config(&chFront);

  ledc_channel_config_t chRear = {};
  chRear.gpio_num   = SERVO_REAR_PIN;
  chRear.speed_mode = LEDC_LOW_SPEED_MODE;
  chRear.channel    = (ledc_channel_t)SERVO_REAR_CH;
  chRear.timer_sel  = LEDC_TIMER_0;
  chRear.duty       = 0;
  chRear.hpoint     = 0;
  ledc_channel_config(&chRear);

  servoWrite(SERVO_FRONT_CH, SERVO_FRONT_PIN, SERVO_DEFAULT_FRONT);
  servoWrite(SERVO_REAR_CH,  SERVO_REAR_PIN,  SERVO_DEFAULT_REAR);
  Serial.println("Servos ready.");


  armESCs();

  lastStatMs = millis();
  lastPktMs  = millis();

}



void loop() {
  static uint32_t lastLoopTime = micros();
  uint32_t now = micros();

  if ((now - lastLoopTime) < LOOP_US) return;
  float dt = (now - lastLoopTime) / 1e6f;
  lastLoopTime = now;
  if (dt > 0.05f) {
    dt = 0.05f;
  }

  if (gotNew){
    consumePacket();
  }
```

```
  if (millis() - lastPktMs > COMM_TIMEOUT_MS) {
    if (ctrl.armed) {
      Serial.println("COMM TIMEOUT — disarming.");
      ctrl.armed = false;
    }
  }


  readIMU(dt);
  Serial.printf("R:%.1f P:%.1f Y:%.1f\n", imu.roll, imu.pitch, imu.yaw);


  if (!ctrl.armed) {
    safeStop();
    printLinkStats();
    return;
  }


  float rollSetpoint    = ctrl.roll  * MAX_ANGLE_DEG;
  float pitchSetpoint   = ctrl.pitch * MAX_ANGLE_DEG;
  float yawRateSetpoint = ctrl.yaw   * MAX_YAW_RATE;


  float rollOut  = computePID(rollSetpoint,    imu.roll,    imu.rollRate,
rollGains,  rollState,  dt);
  float pitchOut = computePID(pitchSetpoint,   imu.pitch,   imu.pitchRate,
pitchGains, pitchState, dt);
  float yawOut   = computePID(yawRateSetpoint, imu.yawRate, 0.0f,
yawGains,   yawState,   dt);

  mixAndWriteMotors(ctrl.throttle, rollOut, pitchOut, yawOut);
  updateServos();

  printLinkStats();
}
```

```
void initESPNow() {
  WiFi.mode(WIFI_STA);
  delay(100);
  WiFi.disconnect();
  delay(100);
  esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE);
  Serial.printf("Drone MAC: %s\n", WiFi.macAddress().c_str());

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init FAILED — restarting.");
    delay(2000);
    ESP.restart();
  }

  esp_now_register_recv_cb(onRecv);
  Serial.println("ESP-NOW receiver ready.");
}



void onRecv(const esp_now_recv_info_t *info, const uint8_t *data, int len)
{
  if (len == sizeof(ControlPacket)) {
    memcpy((void *)&lastPkt, data, sizeof(ControlPacket));
    gotNew = true;
  }
}

void consumePacket() {
  ControlPacket pkt;
  noInterrupts();
  memcpy(&pkt, (void *)&lastPkt, sizeof(pkt));
  gotNew = false;
  interrupts();

  lastPktMs = millis();

  pktCount++;
  pktThisSec++;
  if (firstPkt) {
```

```
    firstPkt = false;
    Serial.println(">>> Link up — first packet received. <<<");
  } else {
    if (pkt.seq != lastSeq + 1)
      dropCount += (pkt.seq - (lastSeq + 1));
  }
  lastSeq = pkt.seq;

  translatePacket(pkt);
}



void translatePacket(const ControlPacket &pkt) {


  if (pkt.buttons & BTN_SELECT) {
    if (ctrl.armed) {
      Serial.println("EMERGENCY STOP — SELECT pressed.");
      ctrl.armed = false;
    }
    rollState  = {0};
    pitchState = {0};
    yawState   = {0};
    ctrl.throttle = 0.0f;
    return;
  }


  bool startNow = (pkt.buttons & BTN_START) != 0;
  if (startNow && !startPrev) {
    ctrl.armed = !ctrl.armed;
    Serial.printf(">>> %s <<<\n", ctrl.armed ? "ARMED" : "DISARMED");
    if (!ctrl.armed) {
      rollState  = {0};
      pitchState = {0};
      yawState   = {0};
      ctrl.throttle = 0.0f;
    }
  }
```

```
startPrev = startNow;


if (pkt.buttons & BTN_UP)
  ctrl.throttle = clampf(ctrl.throttle + THROTTLE_STEP, 0.0f, 1.0f);
else if (pkt.buttons & BTN_DOWN)
  ctrl.throttle = clampf(ctrl.throttle - THROTTLE_STEP, 0.0f, 1.0f);


if (pkt.buttons & BTN_LEFT) {
  ctrl.yaw = -1.0f;
}
else if (pkt.buttons & BTN_RIGHT) {
  ctrl.yaw =  1.0f;
}
else {
  ctrl.yaw =  0.0f;
}



if (pkt.buttons & BTN_TRI)   {
  ctrl.pitch =  1.0f;
}
else if (pkt.buttons & BTN_CROSS) {
  ctrl.pitch = -1.0f;
}
else {
  ctrl.pitch =  0.0f;
}


if{
  (pkt.buttons & BTN_CIRC)  ctrl.roll =  1.0f;
}
else if {
  (pkt.buttons & BTN_SQR)   ctrl.roll = -1.0f;
}
else {
  ctrl.roll  =  0.0f;
```

```
  }

  if {
    (pkt.buttons & BTN_R1) ctrl.armAngle = 1.0f;
  }
  else if (pkt.buttons & BTN_R2) {
    ctrl.armAngle = 0.0f;
  }
}

void printLinkStats() {
  if (millis() - lastStatMs < 1000) return;
  lastStatMs = millis();

  if (pktCount > 0) {
    float dropPct = 100.0f * dropCount / (pktCount + dropCount);
    Serial.printf("--- %lu pkt/s | total: %lu | drops: %lu (%.1f%%) | thr:
%.2f | %s ---\n",
                  pktThisSec, pktCount, dropCount, dropPct,
                  ctrl.throttle,
                  ctrl.armed ? "ARMED" : "disarmed");
  } else {
    Serial.println("--- no packets received yet ---");
  }
  pktThisSec = 0;
}

void readIMU(float dt) {
  sensors_event_t accel, gyro, temp;
  mpu.getEvent(&accel, &gyro, &temp);

  imu.rollRate  = -gyro.gyro.y * 57.2958f;
  imu.pitchRate =  gyro.gyro.x * 57.2958f;
  imu.yawRate   =  gyro.gyro.z * 57.2958f;

  float accelRoll  = -atan2f(accel.acceleration.x, accel.acceleration.z) *
57.2958f;
  float accelPitch = -atan2f(accel.acceleration.y, accel.acceleration.z) *
57.2958f;
```

```
  const float alpha = 0.98f;
  imu.roll  = alpha * (imu.roll  + imu.rollRate  * dt)
            + (1.0f - alpha) * accelRoll;
  imu.pitch = alpha * (imu.pitch + imu.pitchRate * dt)
            + (1.0f - alpha) * accelPitch;


  imu.yaw += imu.yawRate * dt;
}



float computePID(float setpoint, float measured, float measuredRate,
                 PIDGains &g, PIDState &s, float dt) {
  float error   = setpoint - measured;
  float P       = g.Kp * error;
  s.integral    = clampf(s.integral + error * dt, -ITERM_LIMIT,
ITERM_LIMIT);
  float I       = g.Ki * s.integral;
  float D       = -g.Kd * measuredRate;
  return P + I + D;
}



void mixAndWriteMotors(float throttle,
                       float rollOut, float pitchOut, float yawOut) {
  float base = clampf(throttle, 0.0f, MAX_THROTTLE);
  float idleBase = ctrl.armed ? BASE_THROTTLE : 0.0f;
  base = fmaxf(base, idleBase);


  float fl = base + ( rollOut - pitchOut - yawOut) / MIXER_SCALE;
  float fr = base + (-rollOut - pitchOut + yawOut) / MIXER_SCALE;
  float rr = base + (-rollOut + pitchOut - yawOut) / MIXER_SCALE;
  float rl = base + ( rollOut + pitchOut + yawOut) / MIXER_SCALE;

  writeDShot(motorFL, clampf(fl, 0.15f, 0.50f));
  writeDShot(motorFR, clampf(fr, 0.15f, 0.50f));
```

```
  writeDShot(motorRR, clampf(rr, 0.15f, 0.50f));
  writeDShot(motorRL, clampf(rl, 0.15f, 0.50f));
}


void writeDShot(DShotRMT &motor, float val) {
  float pct = clampf(val * 100.0f, DSHOT_MIN_PCT, DSHOT_MAX_PCT);
  if (pct < 1.0f)
    motor.sendThrottle(0);
  else
    motor.sendThrottlePercent(pct);
}


void sendAllDShot(float pct) {
  if (pct < 1.0f) {
    motorFL.sendThrottle(0);
    motorFR.sendThrottle(0);
    motorRR.sendThrottle(0);
    motorRL.sendThrottle(0);
  } else {
    motorFL.sendThrottlePercent(pct);
    motorFR.sendThrottlePercent(pct);
    motorRR.sendThrottlePercent(pct);
    motorRL.sendThrottlePercent(pct);
  }
}


uint32_t degToDuty(int deg) {
  int us = map(deg, 0, 180, SERVO_MIN_US, SERVO_MAX_US);
  us = constrain(us, SERVO_MIN_US, SERVO_MAX_US);
  return (uint32_t)((us / 20000.0f) * 65536.0f);
}

void servoWrite(int channel, int pin, int deg) {
  ledc_set_duty(LEDC_LOW_SPEED_MODE, (ledc_channel_t)channel,
degToDuty(deg));
  ledc_update_duty(LEDC_LOW_SPEED_MODE, (ledc_channel_t)channel);
}
```

```
void updateServos() {
  bool extended = (ctrl.armAngle > 0.5f);
  servoWrite(SERVO_FRONT_CH, SERVO_FRONT_PIN, extended ?
SERVO_FRONT_EXTENDED : SERVO_FRONT_CLOSED);
  servoWrite(SERVO_REAR_CH,  SERVO_REAR_PIN, extended ?
SERVO_REAR_EXTENDED  : SERVO_REAR_CLOSED);
}



void armESCs() {
  Serial.println("Arming ESCs (DShot)...");
  unsigned long t = millis();
  while (millis() - t < 2000) {
    sendAllDShot(0);
    delay(1);
  }
  Serial.println("ESCs armed.");
}



void safeStop() {
  sendAllDShot(0);
  rollState  = {0};
  pitchState = {0};
  yawState   = {0};
}



float clampf(float val, float lo, float hi) {
  if (val < lo) {
    return lo;
  }
  if (val > hi) {
    return hi;
  }
  return val;
```

```
}
```